

REQUEST FOR PRESENTATION

Show & Tell: RunCanon

A portable, self-updating skill management system that observes how teams work with AI agents, discovers reusable workflows, and exports maintainable skill artifacts to Claude, Cursor, Copilot, and beyond.

Request details

PRESENTER

Blessed Uyo

PROJECT

WORKING TITLE

RunCanon

REQUESTED SLOT

TBD, next available date

TIME NEEDED

20-30 min, demo + Q&A

[summary]

I'd like to request a slot at the next Show and Tell to present **RunCanon**, a portable, self-updating skill management system that observes how teams actually work with AI agents, discovers reusable workflows, and generates maintainable skill artifacts for Claude, Cursor, GitHub Copilot, and other harnesses.

The core idea: mine real, structured agent interaction data, language plus ordered tool and prompt sequences, rather than raw UI actions. Score discovered workflows by outcome (success, approval, error rate) rather than a synthetic reward signal. Produce human-readable, version-controllable skill specs that stay under human review by default. Because the system is harness-agnostic, the same mined skill can be exported as a Claude Code skill file, a Cursor rule, a Copilot instruction file, or future formats as they emerge.

[why now]

The inspiration came from a recent paper, *Automating SKILL.md Generation for Computer-Using Agents via Interaction Trajectory Mining* (Hao & Li, 2026, [arxiv.org/pdf/2606.20363](#)). The paper mines GUI trajectories into clustered skills and tries to train a policy from them, and is refreshingly honest about where it falls short.

5 / 8

mined clusters reach ≥ 0.95 purity against ground-truth labels on the source benchmark

18.5% → 20.5%

GRPO-trained skill-step accuracy gain, barely above zero-shot

beaten

a trivial frequency baseline consistently outperforms the trained policy

That negative result is what made this worth pursuing. The paper's failure mode comes from mining raw, unordered GUI clicks and trying to learn an autonomous policy from a synthetic reward model. RunCanon is a different bet on the same underlying question, and it sidesteps the specific weaknesses the paper surfaces.

[where runcanon diverges]

THE PAPER'S LIMITATION	RUNCANON'S APPROACH
- Mines raw, unordered GUI clicks, a bag-of-actions representation that discards sequence	+ Mines language plus ordered tool and prompt sequences, preserving structure from the start
- Trains against a learned, offline reward model that can be gamed or simply uninformative	+ Scores by real outcomes: success, human approval, error rate
- Outputs an opaque, black-box policy that's hard to inspect or correct	+ Outputs human-readable, version-controllable skill specs
- Closing argument: generated artifacts should stay reviewable before deployment	+ Human approval gate on by default; autonomy is opt-in
- Source-domain clusters fail to generalize beyond the original benchmark	+ Harness-agnostic by design, a skill isn't locked to one tool or benchmark

[what i'd cover]

- 01A short walkthrough of the paper's setup and its central negative finding, as framing for why naive trajectory mining isn't enough
- 02The product architecture: the monorepo packages and per-harness export packages
- 03A live look at the current state of the project, with a demo of skill mining and export if it's far enough along by the session date
- 04Open questions and where I'd value feedback, particularly around scoring/lifecycle decisions and harness export fidelity

- @runcanon/core
- @runcanon/spec
- @runcanon/cli
- @runcanon/mcp
- @runcanon/dashboard
- @runcanon/harness-claude
- @runcanon/harness-cursor
- @runcanon/harness-copilot
- @runcanon/harness-openai

[ask]

Could you let me know the next available slot and any format constraints (slide deck, live demo setup, recording policy) I should prepare for? Happy to adjust the scope or length based on what fits the schedule.